

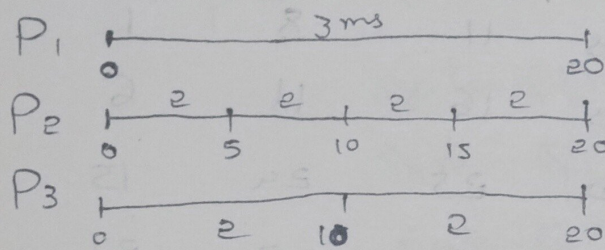
23-3-26

* Real Time OS Scheduling:

① Rate Monotonic Scheduling:

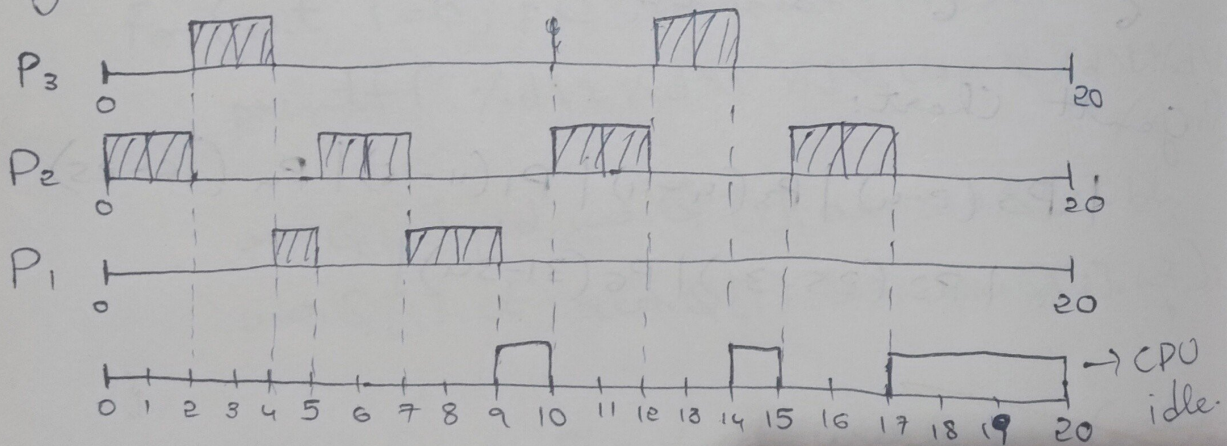
freq. at which each task is scheduled

Periodic Tasks	C	T(Period)	Priority
P ₁	3	20	$\propto \frac{1}{\text{Period}}$
P ₂	2	5	
P ₃	2	10	$\Rightarrow P_2 > P_3 > P_1$



For Gantt chart,
take LCM of all
Periods

Gantt:



```
#include <stdio.h>
```

```
struct task {
```

```
    int id;
```

```
    int exe_time;
```

```
    int period;
```

```
};
```



```

void mono_sche (struct task t[], int n;
                int max-time) {
    for (int i=0; i<n; i++) {
        printf ("Enter task ID, exe-time & period:
                t[i].id, t[i].exe-time, t[i].period);
    }
}

```

```

    for (int i=0; i<n; i++) {
        for (int j=0; j<n-i-1; j++) {
            if (t[i].period > t[j+1].period) {
                struct task temp = t[j];
                t[j] = t[j+1];
                t[j+1] = temp;
            }
        }
    }
}

```

```

int rem_time[n];
int next_arr[n], misses[max-time];
for (int i=0; i<n; i++) {
    rem_time[i] = 0;
    next_arr[i] = 0;
}

```

```

printf ("In Gantt Chart\n");
for (int time=0; time<max-time; time++) {

```

```

    for (int i=0; i<n; i++) {
        if (time == next_arr[i]) {
            rem_time[i] = t[i].cpu;
            next_arr[i] += t[i].period;
        }
    }

```

```

    if (rem_time[i] > 0) {
        misses[time] = t[i].id;
    }
}

```



```

int cur-task = -1;
for(int i = 0; i < n; i++){
    if(rem-time[i] > 0){
        cur-task = i;
        break;
    }
}
if(cur-task != -1){
    printf("T: %d \t", t[cur-task].id);
    rem-time[cur-task]--;
}
else{
    printf("Idle \t");
}
}
printf("\n");
for(int time = 0; time < max-time, time++){
    printf("\t %d", time+1);
}
printf("\n\n Notifications:");
int has-misses = 0;
for(int i = 0; i < max-time; i++){
    if(misses[i] != -1){
        printf("\n [Time: %d] Deadline miss: Task: %d", i, misses[i]);
        has-misses = 1;
    }
}
if(!has-misses) printf("\n No deadlines missed");
printf("\n");
}

```

$$\sum \frac{a_i}{t_i} \leq n(2^{n-1})$$


```

int main() {
    int n, max-time;
    printf("Enter no. of tasks:");
    scanf("%d", &n);
    printf("Enter simulation time:");
    scanf("%d", &max-time);
    struct task t[n];
    for (int i = 0; i < n; i++) {
        printf("Task %d (ID CPU Period):", i+1);
        scanf("%d %d %d", &t[i].id, &t[i].  
exetime  
&t[i].period);
    }
    mono_sche(t, n, max-time);
    return 0;
}

```

3
O/P: Enter no. of tasks: 3
Enter simulation time: 20

Task 1: 1 3 20

Task 2: 2 2 5

Task 3: 3 2 10

Gantt:

|T₂|T₂|T₃|T₃|T₁|T₂|T₂|T₁|T₁|Idle|
0 1 2 3 4 5 6 7 8 9 10

|T₂|T₂|T₃|T₃|Idle|T₂|T₂|Idle|Idle|Idle|
10 11 12 13 14 15 16 17 18 19 20

Enter no. of tasks: 2, simulation time: 10

Task 1: 1 3 5 Task 2: 2 4 8

Gantt chart: |T₁|T₁|T₁|T₂|T₂|T₁|T₁|T₁|T₂|T₂|
0 1 2 3 4 5 6 7 8 9 10

Notifications:

[Time 8] DEADLINE MISS: Task 2